

BitMangler

Mike Havens

Senior Project

Advisor: Dr. Lin

Introduction

BitMangler is an audio plugin utilizing Steinberg's Virtual Studio Technology (VST) interface [1]. The VST standard has been adopted as one of the top three audio plugin formats for digital audio workstations (DAWs). VSTs exist in two distinct genres. The first are synthesizers, which create audio from MIDI, open sound control, and/or user input. The latter are effects, which process and manipulate incoming audio. BitMangler is an effect VST whose primary feature is 'BitMangling', an original take on traditional bitcrushing algorithms. The BitMangler algorithm is detailed later in this document. BitMangler also applies traditional delay and single pole low-pass filter algorithms on the wet signal.

BitMangler was inspired the Harvestman Malgorithm digital/analog effect module [2]. The Malgorithm piqued my interest by bringing the uniquely digital effect of bitcrushing into the realm of modular analog synthesizers. Additionally, the Malgorithm ingeniously used bitwise operations as waveshapes. For example, a signal may be fed back and XORed to itself, shaping the wave in novel ways. Impressed with the Harvestman's craftsmanship, I set out to explore the possibilities of using quantization errors to produce distortion in audio.

The BitMangler Algorithm

To understand the BitMangler algorithm, one must be familiar with how analog audio signals are quantized into digital bytes [3]. Let us use an example recording with an analog to digital converter (ADC) with a sampling frequency of 44.1 kHz and a bit depth of 16 bits. As the motion of our microphone's diaphragm induces current in the instrument's leads, the voltage across the signal transmission wire varies accordingly. The ADC takes a snapshot of this voltage 44,100 times per second. Each of these snapshots are converted into a 16 bit value. The very nature of this conversion involves rounding the continuous signal to its nearest discrete representation.

The output of a traditional bitcrushing algorithm [4] is a representation of the input at the same sample rate, but with a lower bit depth. Each amplitude sample is rounded to its nearest representation using this smaller number of bits. This conversion decreases the digital signal's similarity to the original analog signal, increasing the quantization error. This loss of information effectively introduces new signals, which can either provide warm distortion to the audio, or overpower the signal with dissonant harmonics (waves with non-integer multiples of a note's fundamental frequency). Both of these results are sought after by musicians, such as Daft Punk in their track "Short Circuit" on 2001's "Discovery" (around 2:15 to 3:00) [5].

The BitMangler algorithm is an extension of traditional bitcrushing that is applicable to floating point values. Instead of limiting the number of quantization levels that a user can choose to a power of two, one is free to choose any value that can be realistically represented

with a floating point value by the reciprocal of the quantization level. Currently, the demonstration BitMangler VST hardcodes this value to 64 levels. However, further investigation will be pursued to find maximally pleasing 'warm' and 'harsh' values. The extension of this quantization level beyond powers of two is particularly exciting, as humans process pitch using base two logarithms. Using western tuning as an example, the note A4 has a frequency of 440 Hz. One octave above A4 lies A5 with a frequency of 880 Hz. Expanding quantization levels beyond powers of two has a tendency of introducing inharmonic signals to BitMangled audio. Many musicians prize the technique of creating dissonance by introducing such signals to their music.

Additional Signal Processing

Additional signal processing is included in the BitMangler plugin. First is a delay whose algorithm was provided in the 'adelay' example in Steinberg's VST SDK 2.4. This is a classic delay with a maximum buffer size of 44100 samples, which provides one second of delay buffer for the standard compact disk sampling rate of 44100 kHz. This buffer size is controlled by the delay parameter. The delay's linear decay rate is controlled by the Feedback parameter.

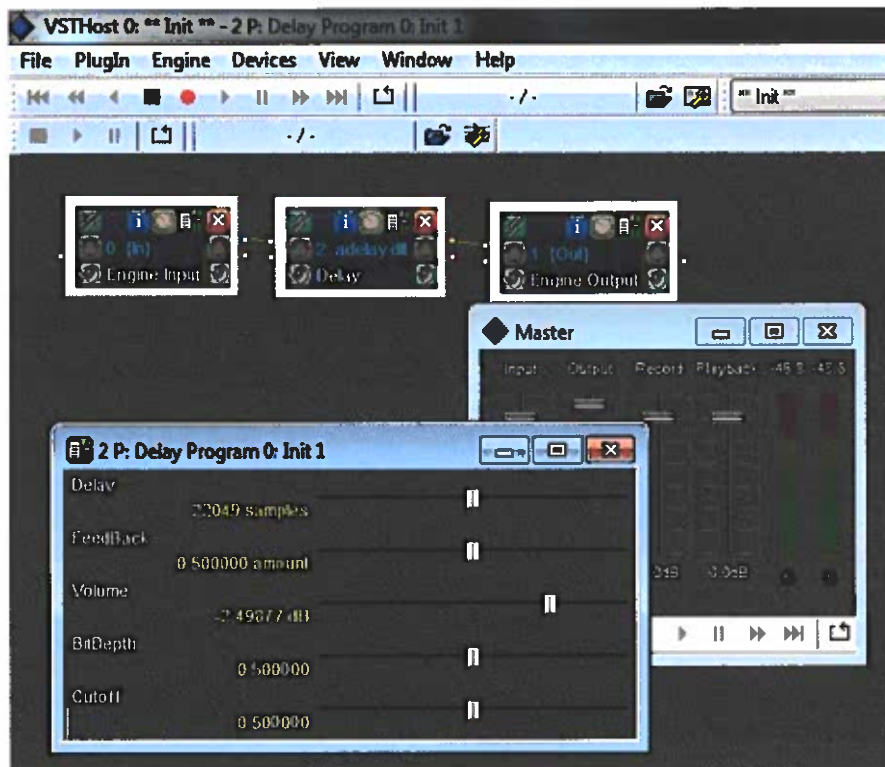
A simple low-pass filter processes the BitMangled audio before it is introduced to the delay. This process is a simple recursive filter that allows the user to filter out the higher pitched harmonic components introduced through BitMangling in order to focus on the lower frequency 'gritty' results. It is of note that this filter is recursive because it uses previous samples in the process of computing a filtered sample's value. This filter does not use recursion in the traditional functional sense of general computer science.

Using BitMangler

The included compact disk contains a zipped folder with this project's source code and compiled VST, found at `\public.sdk\samples\vst2.x\win.vc6\Debug\adelay.dll`. A special thank you is given to Steinberg GmbH for releasing their 'adelay' VST source as documentation to developers. BitMangler used this example from the VST STK 2.4 as a template.

One can place this VST inside of the corresponding plugin folder in their digital audio workstation of choice. Those unfamiliar with digital music may be inclined to use Herman Seibs Hauptseite's VSTHost [6] to evaluate this plugin. Hauptseite has graciously released this plugin under the GNU Lesser GPL license. The below figure is the BitMangler VST running under Hauptseite's VSTHost. The foremost window displays the BitMangler GUI using VSTHost's default style.

This project's source is available at the project's Git repository [7]. Contributions to the code base including both features and bug fixes are welcome.



Citations

- [1] <http://www.steinberg.net/en/company/developer.html>
- [2] <http://www.theharvestman.org/2012.htm>
- [3] <http://www.dspguide.com/ch3.htm>
- [4] <http://www.musicradar.com/tuition/tech/distortion-saturation-and-bitcrushing-explained-549516>
- [5] <http://www.kvraudio.com/forum/printview.php?t=238705&start=0>
- [6] <http://www.hermannseib.com/english/vsthost.htm>
- [7] <https://github.com/mshavens/SeniorProject>